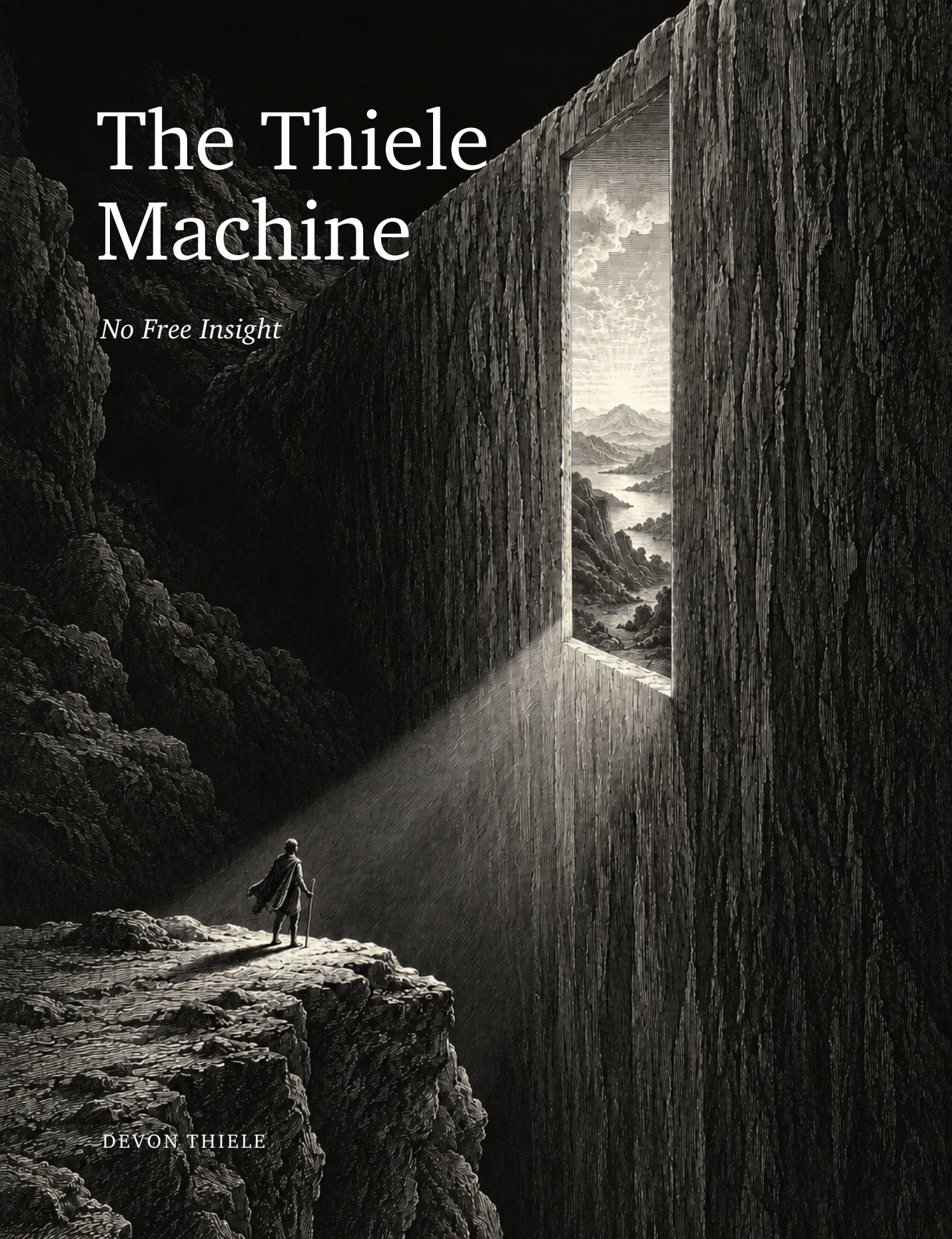
The Thiele Machine

No Free Insight



DEVON THIELE

DEVON THIELE

The Thiele Machine

No Free Insight

The Thiele Machine: No Free Insight

Devon Thiele

Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0).

The source of this book, its proofs and its build are public: github.com/sethirus/The-Thiele-Machine. Archived at doi.org/10.5281/zenodo.17316437.

Set in Charter. Built from its source with pdfTeX.

The cover and part illustrations were generated with AI in the engraving tradition of Gustave Doré. They are new illustrations, not reproductions of his work.

For Julie, and for my dad.

Julie,

Three years ago, a few months into the rendering engine, I came out of my office and told you I thought I was onto something. You said okay. You didn't ask what it was, or what it was for, or when it would be done, and you didn't ask me for anything I could have shown you, because there was nothing to show. There was a program that had done something it shouldn't have been able to do, and a man who didn't yet know what a Turing machine was.

You gave me the three years anyway. The nights after the lot closed. The weekends. The hours I was in the house and not in the house, because my head was in a wall I couldn't get past. Every dead end, and there were more of those than there are pages here. You never once asked me to stop, and you never once said it was fine when it wasn't. You just kept giving me the next evening.

This book is what I did with them. It's the one thing I've built from nothing that I'm sure of, because I threw out every piece of it I couldn't prove, and what's left holds. I know that isn't what you gave the years for. You gave them for me. But it's what I have to show, and I wanted it to be worth showing.

Dad,

The name on the cover was yours first. I asked you before I used it and you said go ahead. You and Mom taught me that nothing is true just because somebody says it is. This book is that lesson, hundreds of pages long. I put our name on it because I wanted one thing in the world with that name on it that nobody handed me, that I couldn't have faked, and that I can stand behind in front of anyone. Here it is.

Julie carries the name now too. So the machine has it third. This is for the two of you who had it first.

Contents

What this is 8

1 Any machine that steps 11

1.1 Is the pencil thing a computer? 11

1.2 How far does that go? 12

1.3 Try to break it 13

1.4 Now ask it about keeping 14

1.5 The machine that stops 14

1.6 What Turing actually proved 15

What this is

I think in shapes, so I'll start with one.

Take a thing. Anything. A coin on a table, a light switch, a page in a notebook. Right now it's one way. It could be another.

Now do something to it. Flip the coin, hit the switch, write on the page. That's a move: a direction that takes the thing from how it is to how it is next.

Here's a computer you can run with a pencil. Write 3 on a scrap of paper. The move is: erase the number and write the next one up. Do it twice. 3, 4, 5. You just added 2 and 3. Nothing happened there but a thing, a move, and the next thing. Hold on to that picture; whether it covers every computer is something I'll have to earn, and I will, later, with you watching.

Here's the part I couldn't let go of. Some moves just shuffle. Flip the coin and you can flip it back, and nothing is different for good. But some moves make a difference that stays. Write a word in pen. Blow a fuse. Check an answer and mark it checked. After that move something is the case that wasn't before, and it stays the case.

I figured there had to be a cost in that step. Not in the shuffling. In the step where something goes from not settled to settled and stays there. That was a hunch, so test it on the pencil computer.

Tell it when to stop: count up, and when you reach 5, stop. Ask one question of each number: is it done? 3, no. 4, no. 5, yes. And once it says yes it keeps saying yes, because the machine has stopped.

A machine that has stopped is still somewhere on the next tick. It stays where it is: 5 goes to 5. Draw the arrows. 4 goes to 5. 5 goes to 5. Two different starting places, one landing place. Standing on 5, you can't tell whether you just got there or have been there all along.

Could you dodge that? Try letting the stopped machine rock back and forth instead of sitting still: 5-tick, 5-tock, 5-tick, both of them saying done. Draw the arrows. 4 goes to 5-tick. 5-tock goes to 5-tick. Two arrows land on 5-tick, the spot where you came in. Rocking only moved the place where they meet. The one way out is to never land anywhere twice: make every tick write something new, a tally mark, the time, anything, so the paper never repeats. That works as long as there's paper. On a finite sheet you run out, and two arrows land on one spot again.

So whenever this machine settles its answer and keeps it, something gives. Either two states become one, or it uses up fresh paper on every tick for as long as it runs.

That part isn't a hunch. You drew it. You drew it for one small counter; the book proves it for any machine that goes from one state to a next, and shows you where.

What I do with it is a choice, and I'll say so: I charge for the move where the answer goes from not done to done. Charging for it is up to me. That something has to give once it makes that move isn't. The two arrows don't always meet on that very move. Give done two spots, 5a and 5b: 4 goes to 5a, 5a goes to 5b, and 5b stays put. Now the arrows from 5a and 5b meet on 5b, a tick after the answer settled. But after the move from not done to done, there's no avoiding it: two arrows meet somewhere, or the paper never repeats.

Two more things have to be pinned down before the rule means anything, and you can see both with what's already on the table.

First, take the coin again. Flip it, flip it back, forever, free. Ask whether it's settled. It never is. A rule that says "a move that settles something has to cost" never asks the coin for anything, because the coin never settles anything. So the rule alone would let a coin that does nothing count. The machine has to actually settle something, at least once.

Second, suppose a later move could pay you back. The move that settles charges you one, the next move hands you one back, and the whole run comes out free. So prices can be nothing or something, never a refund.

One last thing before the whole definition. Settled doesn't have to be just yes or no. Take a checklist with three boxes. None ticked, one, two, all three: each tick settles it a little more, and ticks don't come off. "Done or not" is the smallest case, a checklist with one box. So when I say "more settled" from here on, picture the checklist.

So here's the Thiele Machine, all of it. A thing. Moves that take it to the next thing. A way to tell how settled the thing is, where settled never goes back to unsettled. Some run, from a start you pick, that actually settles it more than it was. A price on every move, nothing or more, never a refund. And one rule: a move that settles something more than it was costs something.

That's the whole definition. The rest of the book works out what follows from it, one step at a time, and tries to break it.

Now look back at the pencil computer with a stop in it. That's Turing's machine, in miniature. Turing's machine, from his 1936 paper, is a tape and a table of rules: from where it is, the table says where it goes next. A thing and moves. It already has something that settles and stays settled: whether it has stopped, the same question you asked of 3, 4 and 5. What Turing never put in is a price on the move that stops it. Put one in, and a Turing machine that ever stops fits every line above. So this isn't a rival to Turing's machine, and it computes nothing his can't. His machine is one kind of Thiele machine, with the price left out.

Now try this. Give two people the same messy list of numbers. The first sorts it, then goes back down it checking that every number is no bigger than the next. The second copies a sorted list off somebody else's sheet. Now look at the two sheets. Same numbers, same order. Look as hard as you like: nothing on either sheet tells you which one was checked. The difference was real, and neither sheet kept it. It's the same thing you saw standing on 5: you can see where you are, and not how you got there.

So put a box on each sheet that says checked. A plain Thiele machine charges for ticking the box, but nothing in the rules above stops the copier from paying and ticking it anyway. Paying for the tick doesn't make it true. So add one more condition: the box can only be ticked right after a check of that very list passes. Now the copier's tick is refused, and a ticked box tells you a check of that list passed. I call a box like that established. It's a condition you put on the machine. The bare definition doesn't hand it to you.

It's called the Thiele Machine. The afterword says why.

CHAPTER 1

Any machine that steps

When I started all this I didn't know what a Turing machine was. I knew it had something to do with computers, and that's it. I had a program that had done something I didn't expect, and a hunch, and no words for either. Most of the words in this book I found the same way: I'd hit a wall, go looking for whoever had been there before me, and not move on until I could watch the idea actually work. This chapter is the first of those trips. I'll take you on it the way I took it.

The question I had was the one I left you with on page one. I'd been calling that pencil-and-scrap-of-paper thing a computer. Is it? Or did I just make it small enough to look like one?

1.1 Is the pencil thing a computer?

Start with what the word used to mean. Before about 1940 a computer wasn't a machine. It was a job. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of what a man was paid. Some of them found things nobody else had; Henrietta Leavitt worked out a way to tell how far away a star is. (I got that from Dava Sobel's history of those women and from the historians at the American Institute of Physics.) Rooms like that worked out artillery tables in both world wars; that one's from David Alan Grier's history of human computers.

So the pencil thing wasn't a toy version of a computer. For a long time it was the only kind there was. Good start, but that's history, not proof. A person with a pencil can do a lot of things that aren't following rules. I wanted to know what was left if you took the person away and kept only the rules.

That's exactly the question Alan Turing asked, in a paper in 1936. Going in, all I knew was that it had something to do with computers. What's in there is a clerk. He wrote down, as plainly as he could, everything a person sitting at a desk with a pencil does when they follow a procedure, and threw out everything else. What was left had four parts, and you can build it on the table in front of you right now.

Draw a strip of three squares. That's the tape. Write 1 in the first square, 1 in the second, leave the third blank.

Put a coin on the first square. That's where the clerk is looking. Turing called it the head.

On a scrap of paper write the letter A. That's which step of the procedure the clerk is on. Turing called it the state.

Now the rules. Write these two on a card:

> If you're on A and looking at a 1: leave it, move the coin one square right, stay on A.
>
> If you're on A and looking at a blank: write a 1 there, don't move, and change the letter to STOP.

That's the whole machine: a tape, a head, a state, and a card of rules. Now run it. Each tick, read the card and do what it says. After each tick, write down everything about the table: what's on each square, where the coin is, and the letter. I'll put brackets around the square the coin is on.

[1]	1	␣	A
1	**[1]**	␣	A
1	1	**[␣]**	A
1	1	**[1]**	STOP

It started with two 1s and ended with three. It added one. That's the same thing the pencil computer did when it went from 4 to 5.

Now look at what you wrote down on each line. The whole tape, where the coin is, and the letter. Is there anything about this machine at that moment that isn't on the line? Try to think of something. The rules don't change. The tape, the coin and the letter are all there is. So one line is the whole thing, and the card takes one line to the next line. Thing, move, next thing.

So the pencil computer wasn't a cheat. Turing's machine is the pencil computer with a bigger piece of paper.

1.2 How far does that go?

It would be nice if the answer were "every computer, done." It isn't quite. Push on it and it came apart into three different claims that people tend to run together. Here they are, and what each one stands on.

The first one is the one we just did. Every machine you can write down as a whole state and a rule for the next state has the shape from page one. You checked that yourself for Turing's machine: one line, one card, next line.

The second is that the other ways people have written computation down, with registers instead of a tape, or with functions handed to functions (that's Alonzo Church's lambda calculus, from the same year as Turing's paper), compute exactly the same things Turing's machine does. I didn't prove that, and I won't pretend I did. Other people proved it, one pairing at a time; Turing himself sketched the

match with Church's version in an appendix to his 1936 paper. What you can check yourself is that each of those is also a whole state and a rule for the next one, the same way you just did with the coin.

The third is the big one, and here's where I had to be careful. Does this cover everything a person could ever do by following rules? Does it cover every machine anybody could ever build out of real stuff? Those turned out to be claims nobody can prove, and for a good reason. "Everything a person could do by following rules" isn't a precise enough thing to prove anything about. That claim has a name, the Church-Turing thesis. "Every machine you could build" is a claim about physics, and the only way to knock it down would be to build one that beats Turing's, which nobody has done. Neither is a theorem, and in this book neither will get treated like one.

So the debt from page one comes out like this. For every machine you can write down as a whole state and a rule for the next state, paid: you just checked it. For every machine anybody could ever build, that's a claim about the world, it's labeled as a claim, and I'm not going to lean on it.

1.3 Try to break it

Now the question that actually mattered to me. Page one showed that when the counter settled on 5, two arrows had to meet, unless it used fresh paper forever. That was one little counter. Is it true of every machine with this shape?

Don't take my word for it. Get a sheet of paper and do this.

Draw some dots. Ten, twenty, as many as you like. Those are states. From every dot, draw exactly one arrow to some dot. It can point anywhere, even back at the dot it starts from. That's the one rule: every state has exactly one next state.

Now pick a dot to start on and follow the arrows. Keep a list of every dot you've been on.

Do it a few times, with different drawings and different starts. Watch what happens at each step. You'll find that each step does one of three things.

It lands on a dot that isn't on your list yet. A new one.

It lands on the dot you started from. Now you're going around in a circle, and since every dot has only one arrow out, you'll go around that same circle forever.

Or it lands on some other dot that's already on your list. Look at the first time that happens. The first time you got to that dot, you came in on an arrow from the dot right before it on your list. Now you've come in again, on an arrow from the dot you were just on. Those have to be two different dots, because until this step you hadn't been on any dot twice. Two arrows, from two different dots, landing on one. That's a merge, the same thing as 4 and 5 both going to 5. And from there you're on a path you've walked before, so you walk it again, forever.

Now try to make it do a fourth thing. Really try. Draw whatever you want.

Here's why you can't. Wherever the next arrow points, that dot is either on your list or it isn't. If it isn't, it's new. If it is, it's either the start or it isn't the start. There's no other place for the arrow to point. Three cases, and they cover everything.

And notice what you never had to know: how many dots there were. Ten dots or a billion or an endless sheet, it's the same three cases. What you did need was that every dot has exactly one arrow out. Hold on to that, because it's about to matter.

1.4 Now ask it about keeping

Go back to your dots. Color the dot you start on white. Pick some other dots and color them black, and make one rule about the coloring: no arrow is allowed to go from a black dot to a white one. Once you're on black, you stay on black. That's "settled stays settled." White is not done, black is done.

Follow a run from your white start until it lands on black. Now which of the three things can happen after that?

It can't circle back to the start. The start is white, and you can never get back to white once you're on black.

So it's one of the other two. Either it keeps landing on new dots forever, or two arrows meet somewhere.

Try to draw a counterexample: a run that goes from white to black, stays black, never lands on a new dot forever, and never has two arrows meet. You'll be drawing for a while. There isn't one, and the three cases above are the reason.

That's what page one promised: "the book proves it for any machine that goes from one state to a next." You drew it for one counter there. Now you've drawn it for any machine at all, any number of dots.

What it doesn't tell you is anything about heat. Whether two arrows meeting gives off heat in a real device is a question about the device. People argue about it, and they get their own chapter. This is dots and arrows and nothing else.

1.5 The machine that stops

Here's where the "exactly one arrow out" part bites.

Go back to the tape and the coin. The last line was 1 1 [1], STOP. What does the card say to do next? Nothing. There's no rule for STOP. The machine just stops. In dots and arrows: the STOP dot has no arrow out.

That breaks the one thing the three cases needed. So try the keeping question on a machine that stops. Draw two dots. "Working" is white. "Stopped" is black. One arrow, from working to stopped, and no arrow out of stopped.

Follow it. White, then black, then nowhere. Did two arrows meet? No, there's only one arrow. Did it use up new dots forever? No, it stopped. The answer settled and nothing gave.

Now ask it the obvious question: what does a stopped machine do on the next tick? Not "nothing happens," because something is still there. The usual answer is that it stays where it is. So give stopped an arrow back to itself. Now look again. Working points to stopped. Stopped points to stopped. Two arrows, one dot. The merge is back.

What if you don't want it to stay put? Point stopped at a new dot, "parked," and point parked at itself. Now stopped and parked both point at parked. Two arrows, one dot again. The merge moved one dot over. It didn't go away. That's the rocking machine from page one.

So a machine that stops only dodges this by not answering what happens next. Answer it any way you like, and you're back in the three cases: either two arrows meet, or you draw new dots forever.

1.6 What Turing actually proved

Almost every account you'll find says Turing proved the famous halting problem: that no machine can look at every other machine and say whether it stops. Since the stopping machine is right in the middle of this chapter, I went and looked at what he actually wrote, with an AI walking me through the paper and the links it turned up, and it isn't quite what happened.

Turing's own machines in 1936 weren't built to stop at all. The ones he cared about printed out the digits of a number, forever. What he proved no machine could always decide was a question about those: whether a machine keeps printing digits, and whether it ever prints a given symbol. The second one is a few lines away from the halting problem, so in a real sense he had it. But he never wrote it down that way. Stephen Kleene stated it in 1952. Martin Davis seems to be the first to have called it "the halting problem," in 1958. I found that in a paper where Joel David Hamkins and Theodor Nenu went back to Turing's 1936 paper and checked, line by line.

So the machine that stops, the one sitting right in the hole, is Turing's machine the way people later learned to read it. It's the one page one meant when it said Turing's machine already has something that settles and stays settled: whether it has stopped.

Put it together. Read a Turing machine through "has it stopped?" Give the stopped line a next line by letting it stay put. Then the step that stops it is a step where two arrows meet, which you just drew. That's the step I charge for. Charging for it is still my choice. That something gives there isn't.

The dots in this chapter could go on forever. The next question is what happens when they can't, when the paper runs out.